INTERN TASK REPORT

Task:	Task 3.4 — Interview Scheduling System & Email Notification System
Intern Name:	Jyoshna Sankarapu
Domain:	AI / ML
Date of Task Given:	11/08/2026
Captain:	Aditya Kanojiya
Team Members:	Himanshi, Meghana, Nandini Singh, Jyoshna Sankarapu, Niharika

1. Task Overview

Task Description: The assigned task was to implement Task 3.4 — Interview Scheduling System with an AI/ML Email Notification Engine in the intelliview-orchestrator repository. The task required building an automated smtplib-based email notification module, backend scheduling API endpoints, database ORM models, an interactive Next.js calendar view, and overview dashboard widgets.

Objective: The objective was to enable HR managers to pick a candidate, date/time, and interviewer to schedule interviews in advance. Upon successful schedule creation, the system automatically triggers an email notification (using Python's smtplib and email.mime) to the candidate containing complete interview details, while maintaining proper error handling to prevent API failures if the email server is offline.

2. Technologies Used

- **Python 3.11 / 3.13:** smtplib, email.mime, FastAPI, SQLAlchemy ORM, Pydantic, Pytest
- **Next.js & React:** Modern calendar grid view, SWR data fetching, Tailwind CSS, Lucide Icons
- **Code Quality & CI Linters:** Black, isort, Ruff
- **Version Control:** Git & GitHub (Branching, line-ending normalization via .gitattributes, single-commit squashing)
- **Testing Environment:** Pytest (Isolated unit & mock API integration testing)

3. Implementation

The following activities were performed to complete the task:

- **Repository Setup & Branching:** Forked and cloned the repository, checking out the *Stabilized-version* branch, and added team members as collaborators.
- **Database ORM Model (InterviewSchedule):** Created *database/models/interview_schedule.py* mapping the *interview_schedules* table with fields id, candidate_id, interviewer_id, scheduled_at, status, notes, created_at, and updated_at.
- **Email Notification Engine (smtplib):** Created *orchestrator/email_service.py* using Python's standard smtplib and email.mime to send HTML and text confirmation emails containing Candidate Name, Email, Date, Time, Interviewer, and Schedule Reference ID.
- **Backend API Endpoints (/api/schedule):** Implemented *routers/schedule.py* providing POST (schedule & dispatch email), GET (list schedules), GET /upcoming (upcoming events), and PATCH (status updates) endpoints.
- **Frontend Calendar View & Dashboard Widgets:** Built an interactive calendar UI and HR scheduling form at *frontend/src/app/schedule/page.jsx*, and added an Upcoming Scheduled Interviews widget to the main Overview Dashboard.

• **CI/CD Compliance & Line Ending Normalization:** Created *.gitattributes* enforcing Unix LF line endings to resolve Linux runner Black format check failures, and executed formatting fixes across all files.

• **Testing & Single Commit PR Preparation:** Added unit test suite in *tests/test_interview_schedule.py* (14/14 tests passed). Squashed all modifications into a single clean commit (e33b4614) and force-pushed to origin.

4. Challenges Faced

Challenge	Solution
Linux CI Black Format Failure: Windows CRLF line endings caused black --check . to fail on Ubuntu GitHub Actions runners.	Created *.gitattributes* enforcing **.py text eol=lf* and converted line endings of all Python files to LF.
SMTP Server Unavailability: External SMTP servers may be unreachable in dev environments.	Wrapped SMTP dispatch in try/except *smtplib.SMTPException* catching with logging fallback so scheduling API requests succeed even if SMTP server is offline.
Timezone Alignment: ISO datetime strings sent from frontend lost timezone context.	Enforced explicit UTC timezone conversion (*scheduled_at.replace(tzinfo=timezone.utc)*) prior to DB storage and email formatting.
Single Commit Requirement: Incremental commits violated the single-commit PR rule.	Used *git reset --soft* to squash all changes into a single clean commit (*e33b4614*).

5. Outcome

• **Email Notification System:** Successfully built and integrated smtplib notification engine that sends candidate confirmation emails upon scheduling.

• **Full-Stack Functionality:** Fully functional Next.js calendar scheduling interface (/schedule) and dashboard overview widgets.

• **Code Quality & CI:** 100% of unit tests passed (14/14 tests), and GitHub Actions CI checks (CI / Python tests, CI / Frontend, CI / Docker build validation) passed with green checkmarks.

• **Single Commit Compliance:** Squashed into 1 clean commit (e33b4614) ready on Stabilized-version branch.

6. Learning Outcomes

• Developing automated email notification services using Python's standard smtplib and email.mime modules.

• Integrating database ORM models (SQLAlchemy) and FastAPI route handlers with React / Next.js UI components.

• Troubleshooting cross-platform line ending issues (Windows CRLF vs Linux LF) in GitHub Actions CI pipelines.

• Following strict open-source pull request guidelines, including Black formatting, Ruff linting, and single-commit squashing rules.

7. Conclusion

This task provided valuable experience in building full-stack interview orchestration features and email notification pipelines. I learned how to handle asynchronous/SMTP side effects safely in FastAPI endpoints, build interactive calendar interfaces, resolve Linux CI build formatting issues, and maintain high code quality standards.

GitHub Repository: https://github.com/sankarapujyoshna-collab/intelliview-orchestrator/tree/Stabilized-version

Pull Request Link: https://github.com/rajat-wyrm/intelliview-orchestrator/compare/Stabilized-version...sankarapujyoshna-collab:intelliview-orchestrator:Stabilized-version